

Chapter 3: Methodology

Explainable Predictive Maintenance Framework for MetroPT-3

3.1 Introduction

This chapter presents the methodology for designing and evaluating an explainable machine learning framework for early failure detection in a metro train air compressor. The project uses the MetroPT-3 dataset to test whether operational sensor readings can be transformed into early-warning predictions, interpretable explanations and provisional maintenance risk outputs.

The methodology addresses temporal dependency, rare failure events, early-warning label design, class imbalance, model interpretability and decision support. It therefore avoids random train-test splitting and accuracy-led evaluation. Instead, it uses chronological validation, leakage-safe feature engineering, predetermined threshold selection, event-level evaluation, SHAP-based explanation and provisional risk-level interpretation.

3.2 Research Philosophy and Design

This dissertation adopts a quantitative, empirical, secondary-data computational design using comparative predictive modelling. No compressor manipulation or treatment allocation occurs, so the study is not experimental in a laboratory sense.

The research position is post-positivist because the compressor has a real physical condition, but this condition is observed indirectly through sensor readings that may contain noise, uncertainty or missing values. This requires cautious validation, transparent uncertainty reporting and avoidance of unsupported causal claims.

The output is a reproducible Python-based framework for data loading, preprocessing, feature engineering, early-warning labelling, modelling, evaluation, explainability and provisional risk categorisation. This aligns with predictive maintenance research, where data-driven methods are used to identify developing faults before operational disruption (Carvalho et al., 2019; Zonta et al., 2020).

3.3 Dataset Description and Data Management

MetroPT-3 contains multivariate time-series readings from the Air Production Unit of a metro train compressor. The UCI repository reports 1,516,948 records and 15 features, including pressure, oil temperature, motor current, compressor state and digital valve or switch indicators (Davari et al., 2021). It is suitable because it is real industrial data with documented failure intervals, temporal dependence and severe class imbalance. Veloso et al. (2022) further describe the MetroPT data as a railway compressor predictive maintenance dataset.

The raw dataset will be kept separate from processed files so that transformations are reproducible. During implementation, processed data, features, models, outputs and figures will use clear file names. GitHub will initially hold documentation, dataset instructions and the implementation roadmap; code, notebooks and outputs will be added as produced.

Table 3.1: Dataset variables and methodological use.

Variable group	Variables	Use in methodology
Time index	timestamp	Chronological sorting, splitting and warning-window labels
Pressure sensors	TP2, TP3, H1, DV_pressure, Reservoirs	Raw, rolling and pressure-difference features
Thermal/load sensors	Oil_temperature, Motor_current	Raw, lag, rolling and abnormal-behaviour features
Digital signals	COMP, DV_electric, Towers, MPG, LPS, Pressure_switch, Oil_level	Compressor and valve operating-state indicators
Flow signal	Caudal_impulses	Air-flow related operational feature

Table 3.2: Reproducibility and data management requirements.

Area	Procedure
Dataset source	UCI MetroPT-3 public dataset; source URL and access date recorded
Raw data	Stored locally, retained unchanged and excluded from GitHub if too large
Processed data	Cleaned data, engineered features and labels saved separately
Environment	Python 3.11 planned; package versions frozen in requirements.txt or environment.yml
Operating details	Operating system, hardware details and random seed reported in final appendix
Version control	GitHub repository with meaningful commits as implementation progresses
Evidence	Final code, notebooks, figures, results and repository link included after implementation

3.4 Methodological Workflow

The project follows an end-to-end workflow. Raw records are inspected, cleaned and explored before time-series features and early-warning labels are created. Models are trained using chronological validation, evaluated using record-level and event-level metrics, explained using SHAP and translated into provisional maintenance risk categories.

Figure 3.1 Overall methodology workflow

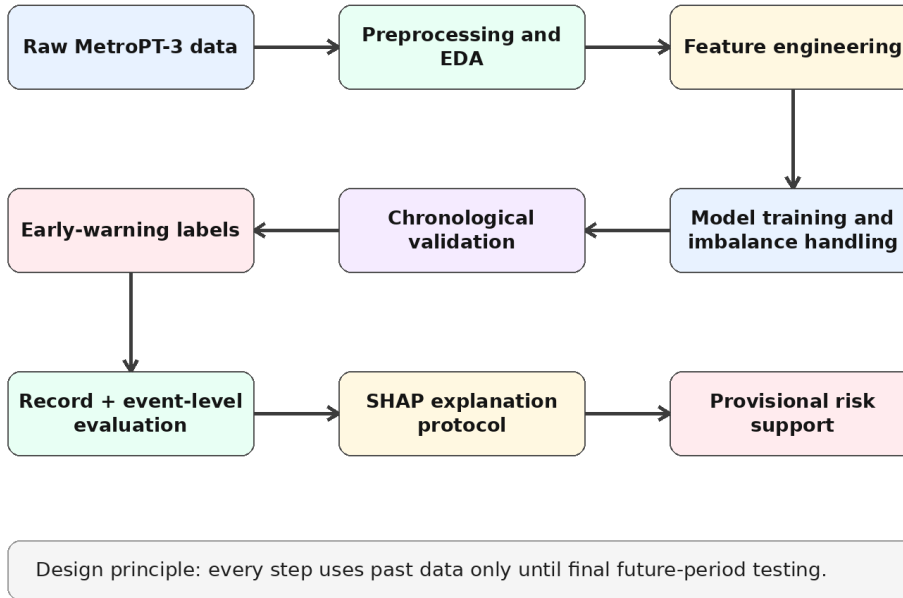


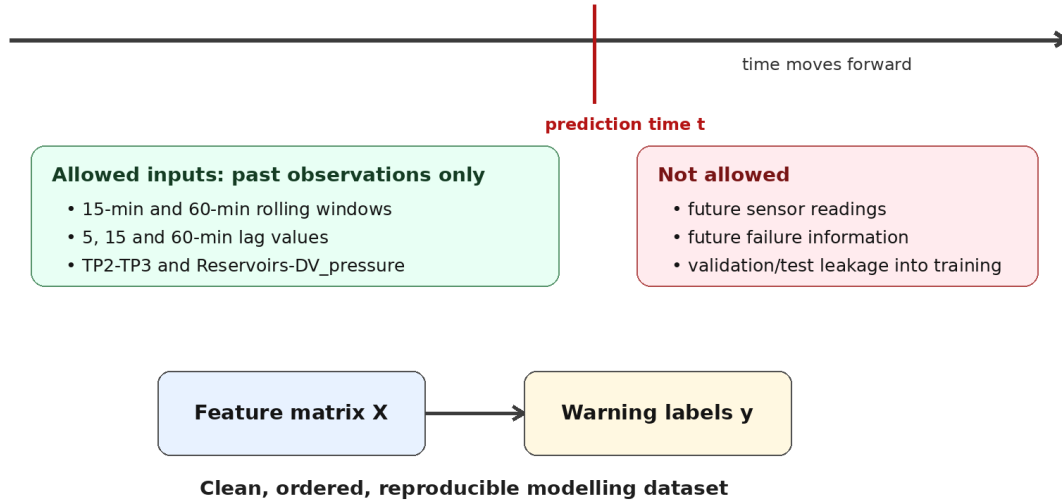
Figure 3.1: Overall methodology workflow for the explainable predictive maintenance framework.

3.5 Data Preprocessing and Exploratory Analysis

The dataset will be inspected for variable names, data types, timestamp range, sensor structure, duplicate timestamps and missing values. The timestamp column will be converted into datetime format and all records sorted chronologically because later modelling steps depend on temporal order. If missing values are small and occur within continuous sensor streams, forward filling may be applied cautiously; no imputation will use future observations.

Continuous variables such as pressure, oil temperature and motor current will be retained as numerical features, while digital compressor and valve states will be retained as binary indicators. Outliers will not be automatically removed because extreme values may represent abnormal compressor behaviour. Only impossible import or sensor-recording errors will be corrected or removed.

Exploratory analysis will use summary statistics, time-series plots, distribution plots and correlation analysis. The purpose is to guide feature engineering and interpretation, not to claim causal fault mechanisms.

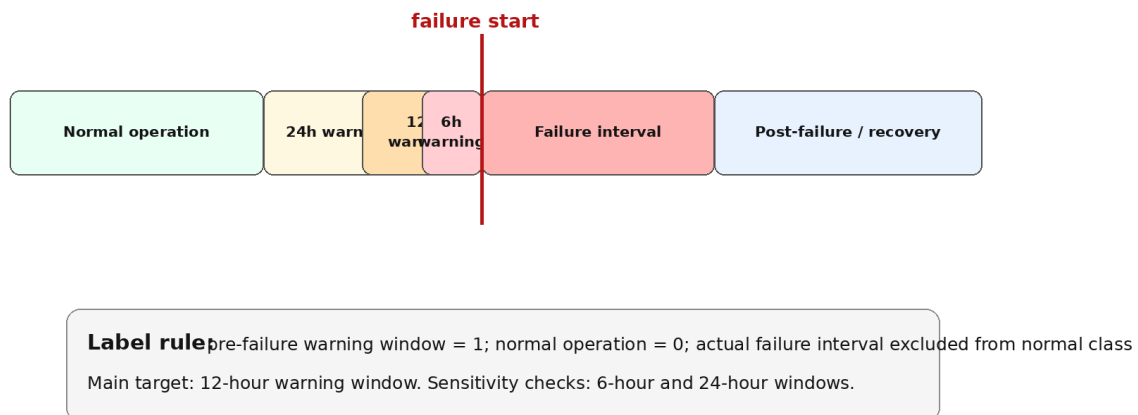
Figure 3.2 Leakage-safe preprocessing and feature engineering*Figure 3.2: Leakage-safe preprocessing and time-series feature engineering design.*

3.6 Feature Engineering and Early-Warning Labels

Feature engineering will capture temporal behaviour that may not be visible in individual sensor readings. The main numerical sensors used for engineered features are TP2, TP3, H1, DV_pressure, Reservoirs, Oil_temperature, Motor_current and Caudal_impulses.

Rolling-window features will be calculated using time-based 15-minute and 60-minute windows. For each selected numerical sensor, rolling mean, rolling standard deviation, rolling minimum, rolling maximum and rate of change will be created. The 15-minute window captures short-term instability; the 60-minute window captures slower drift. Lag features will be created at 5-minute, 15-minute and 60-minute intervals. Pressure-difference features will include TP2 minus TP3 and Reservoirs minus DV_pressure. All rolling and lag features will use past observations only.

The target variable is designed for early-warning prediction rather than post-failure detection. Observations before each documented failure are labelled as warning cases. The 12-hour warning horizon is the primary target because it balances maintenance usefulness and detectable signal. Six-hour and 24-hour windows will be used for sensitivity analysis. Actual failure intervals are excluded from the normal class because the aim is to predict the pre-failure period rather than recognise the compressor after failure has already occurred.

Figure 3.3 Early-warning label construction*Figure 3.3: Early-warning label construction around documented failure intervals.*

Chapter 3: Methodology - Explainable Predictive Maintenance Framework
Table 3.3: Documented MetroPT-3 failure intervals and modelling treatment.

Event	Failure type	Failure interval	Primary label treatment
F1	Air leak	18 Apr 2020 00:00 - 18 Apr 2020 23:59	Pre-failure warning windows labelled positive; failure interval excluded from normal class
F2	Air leak	29 May 2020 23:30 - 30 May 2020 06:00	Pre-failure warning windows labelled positive; failure interval excluded from normal class
F3	Air leak	05 Jun 2020 10:00 - 07 Jun 2020 14:30	Validation event; threshold and calibration decisions based on validation period
F4	Air leak	15 Jul 2020 14:30 - 15 Jul 2020 19:00	Final future-period test event where chronologically appropriate

3.7 Data Splitting and Event-Level Validation Strategy

The dataset will be split chronologically rather than randomly. Random splitting is unsuitable because neighbouring time-series observations are highly related and may create optimistic results if similar operating periods appear in both training and testing data. Chronological splitting reflects deployment, where a model trained on past behaviour must predict future compressor behaviour (Bergmeir, Hyndman and Koo, 2018; Hyndman and Athanasopoulos, 2021).

Because MetroPT-3 contains only four documented failure events, the methodology prioritises event-level validation. The planned main partition is: training from the dataset start to 31 May 2020 23:59, validation from 1 June 2020 00:00 to 30 June 2020 23:59, and final testing from 1 July 2020 00:00 to the dataset end. This places F1 and F2 in training, F3 in validation and F4 in the future-period test set. Where feasible, an expanding-window check will also be reported by training on earlier events and evaluating on the next later event.

The final test period remains untouched until model, threshold and calibration decisions are complete.

Figure 3.4 Chronological partition and event-level validation

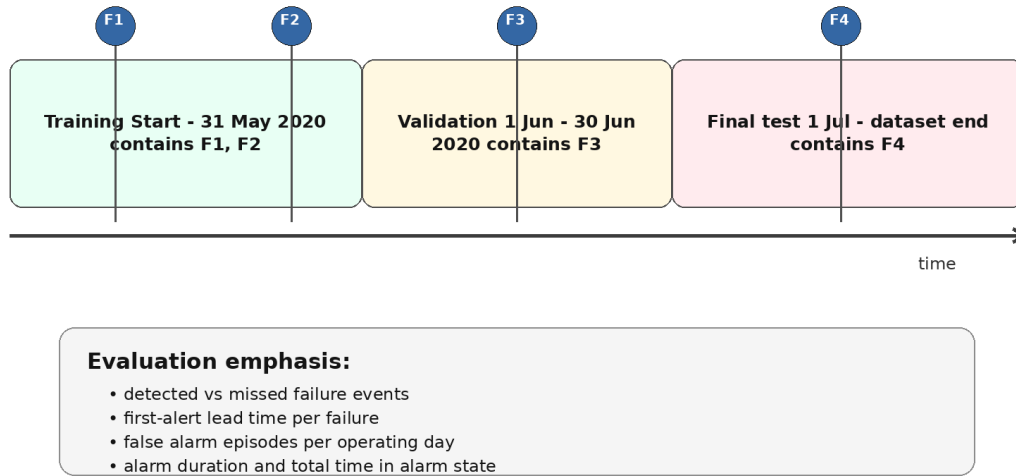


Figure 3.4: Chronological hold-out partition and event-level validation strategy.

3.8 Class Imbalance Handling and Model Development

Failure-warning observations represent a very small proportion of the data, which is typical because equipment normally operates without failure for long periods (Carvalho et al., 2019). Three strategies will be used: class weighting, optional training-only SMOTE and threshold tuning. SMOTE will be applied only after chronological splitting and only to the training set, because oversampling must be handled carefully in imbalanced learning (Fernandez et al., 2018). It will not be applied to validation or test data.

Four models will be compared. Logistic Regression provides a transparent baseline. Random Forest and XGBoost test stronger nonlinear supervised models. Isolation Forest is included as an anomaly-detection comparator because rare failures may appear as abnormal behaviour. Hyperparameter tuning will use validation data only; the test set will not be used for model selection.

Table 3.4: Model hyperparameter search space.

Model	Hyperparameters considered	Purpose
Logistic Regression	C: 0.01, 0.1, 1, 10; penalty: l2; solver: liblinear/saga; class_weight: balanced/none	Baseline regularisation and imbalance handling
Random Forest	n_estimators: 100, 300, 500; max_depth: None, 10, 20; min_samples_leaf: 1, 5, 10; max_features: sqrt/log2; class_weight: balanced/none	Controls ensemble size, complexity and class weighting
XGBoost	n_estimators: 100, 300, 500; learning_rate: 0.01, 0.05, 0.1; max_depth: 3, 5, 7; subsample: 0.8, 1.0; colsample_bytree: 0.8, 1.0; scale_pos_weight: class-ratio based	Controls boosting complexity, overfitting and minority weighting
Isolation Forest	n_estimators: 100, 300, 500; max_samples: auto/0.8; contamination: validation-based range; max_features: 0.8, 1.0	Controls anomaly sensitivity and sampling behaviour

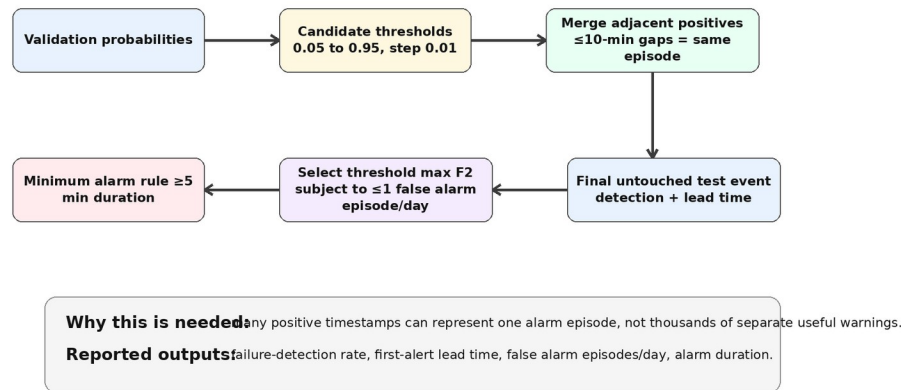
3.9 Model Evaluation, Thresholding and Calibration

Evaluation will use both record-level and event-level metrics. Record-level metrics include precision, recall, F1-score, F2-score and Precision-Recall AUC. Accuracy will be reported only as a secondary metric because it can be misleading when normal observations dominate. PR-based evaluation is useful for minority-class warning performance under imbalance (Williams, 2021).

Record-level metrics are insufficient because adjacent positives may form one continuous alarm, not many useful warnings. Therefore, positive predictions will be merged into alarm episodes. Consecutive positive predictions separated by gaps of no more than 10 minutes will be treated as the same alarm episode. A valid alarm must last at least 5 minutes. Episodes shorter than 5 minutes will be ignored unless they occur within a documented warning window.

A failure event is counted as detected if at least one valid alarm episode begins within the selected warning window before the failure start time. Event-level evaluation will report detected/missed failures, detection rate, first-alert lead time, lead-time range, false alarm episodes per day, alarm duration and total alarm time.

Threshold selection will be predetermined before final testing. Candidate thresholds from 0.05 to 0.95 in 0.01 steps will be evaluated on validation data. The selected threshold will maximise validation F2 while allowing no more than one false alarm episode per operating day. If several thresholds satisfy this condition, the threshold with the highest event-level failure-detection rate will be selected; if still tied, the longer median lead time will be chosen. Probability calibration will be fitted using validation data only because risk categories depend on meaningful probability estimates (Silva Filho et al., 2023).

Figure 3.5 Threshold selection and alarm episode evaluation**Figure 3.5: Threshold selection and alarm episode evaluation pathway.****Table 3.5: Record-level and event-level evaluation metrics.**

Metric type	Metric	Purpose
Record-level	Precision, recall, F1-score, F2-score	Measures timestamp-level warning quality; F2 prioritises recall
Record-level	PR-AUC	Evaluates ranking under class imbalance
Event-level	Detected/missed events and failure-detection rate	Shows whether each documented failure was captured
Event-level	First-alert lead time and lead-time range	Measures how early useful warning was produced
Alarm-level	False alarm episodes per operating day	Measures operational false alarm burden
Alarm-level	Alarm duration, total alarm time and alert-episode precision	Measures maintenance workload and episode usefulness

3.10 Explainability and Decision-Support Framework

SHAP will explain the selected supervised model using a fixed protocol. If XGBoost or Random Forest is selected, TreeExplainer will be used. If Logistic Regression is selected, LinearExplainer will be used. Isolation Forest will remain an anomaly-detection comparator and will not be the main SHAP-explained decision-support model unless it clearly outperforms the supervised alternatives.

The SHAP background dataset will come from the training period only to avoid leakage. Up to 1,000 training observations will be used as background. Global explanations will use up to 2,000 validation/test observations, including all warning cases where possible plus sampled normal cases. Local explanations will include true positives, false positives and false negatives, because missed warnings are safety-relevant and must be reviewed alongside successful and incorrect alerts.

Correlated rolling and lag features will be interpreted by grouping them into sensor families such as pressure, temperature, current and engineered temporal features. SHAP values will not be interpreted causally. A high SHAP value means that a feature influenced the model prediction; it does not prove that the sensor characteristic physically caused the failure. Explanation stability will be checked, where feasible, by comparing whether similar feature groups remain influential across temporal validation folds (Arrieta et al., 2020; Molnar, 2022; SHAP Developers, 2026).

The final framework will translate predictions into provisional decision-support information: warning probability, main explanatory feature groups and a risk category from Normal to Critical. These categories are research outputs only and are not validated operational maintenance rules. They would require review by qualified railway maintenance engineers before real-world deployment.

Figure 3.6 SHAP explanation and provisional risk-support protocol

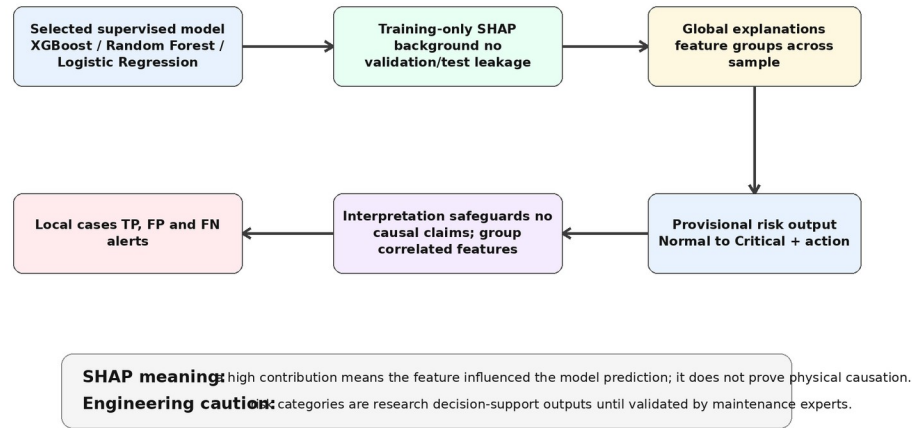


Figure 3.6: SHAP explanation and provisional risk-support protocol.

Table 3.6: Provisional risk categories and recommended maintenance actions.

Risk category	Interpretation	Recommended action
Normal	Sensor behaviour consistent with normal operation	Continue normal monitoring
Low	Minor increase in predicted warning probability	Continue operation and monitor trend
Medium	Developing abnormality in pressure, temperature or current	Schedule closer inspection
High	Strong warning probability with supporting explanation	Prioritise maintenance inspection
Critical	Very high or repeated high-risk alerts	Urgent engineering review

3.11 Software Environment, Ethics and Limitations

The project will use Python 3.11, Jupyter Notebook and a fixed random seed of 42 where supported. Pandas, NumPy, scikit-learn, XGBoost, imbalanced-learn, matplotlib, seaborn and SHAP will support processing, modelling, oversampling, visualisation and explainability. The final repository will include requirements.txt or environment.yml, dataset download information, package versions, operating environment and hardware details.

The project uses public secondary machine data and does not involve human participants, interviews, questionnaires or personal data. The main ethical issues are correct data use, accurate citation, academic integrity and responsible interpretation. Results will report false positives, false negatives and limitations because overstating reliability in a railway maintenance context would be professionally irresponsible.

The main limitation is the small number of documented failure events. Although the dataset contains many timestamped observations, only four failure events restrict generalisability. Early-warning labels also depend on the selected horizon, so 6-hour, 12-hour and 24-hour windows will be compared. Finally, oversampling may not preserve temporal structure, and the risk categories require engineering validation before operational use.

3.12 Chapter Summary

This chapter presented a replicable methodology for an explainable predictive maintenance framework using MetroPT-3. The revised design specifies preprocessing rules, feature windows, lag intervals, date-based partitions, hyperparameter ranges, threshold-selection criteria, event-level metrics, alarm-episode rules, SHAP protocol, reproducibility requirements and provisional risk categorisation. These choices directly address temporal leakage, class imbalance, rare failure events, interpretability and operational usefulness.

References

- Arrieta, A.B., Diaz-Rodriguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., Garcia, S., Gil-Lopez, S., Molina, D., Benjamins, R., Chatila, R. and Herrera, F. (2020) 'Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI', *Information Fusion*, 58, pp. 82-115.
- Bergmeir, C., Hyndman, R.J. and Koo, B. (2018) 'A note on the validity of cross-validation for evaluating autoregressive time series prediction', *Computational Statistics & Data Analysis*, 120, pp. 70-83.
- Carvalho, T.P., Soares, F.A.A.M.N., Vita, R., Francisco, R.P., Basto, J.P. and Alcala, S.G.S. (2019) 'A systematic literature review of machine learning methods applied to predictive maintenance', *Computers & Industrial Engineering*, 137, article 106024.
- Davari, N., Veloso, B., Ribeiro, R. and Gama, J. (2021) MetroPT-3 Dataset. UCI Machine Learning Repository. doi: 10.24432/C5VW3R. Available at: <https://archive.ics.uci.edu/dataset/791/metropt+3+dataset> (Accessed: 4 August 2026).
- Fernandez, A., Garcia, S., Herrera, F. and Chawla, N.V. (2018) 'SMOTE for learning from imbalanced data: progress and challenges, marking the 15-year anniversary', *Journal of Artificial Intelligence Research*, 61, pp. 863-905.
- Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice*. 3rd edn. Melbourne: OTexts.
- Molnar, C. (2022) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd edn. Munich: Christoph Molnar.
- SHAP Developers (2026) SHAP documentation. Available at: <https://shap.readthedocs.io/> (Accessed: 4 August 2026).
- Silva Filho, T., Song, H., Perello-Nieto, M., Santos-Rodriguez, R., Kull, M. and Flach, P. (2023) 'Classifier calibration: a survey on how to assess and improve predicted class probabilities', *Machine Learning*, 112, pp. 3211-3260.
- Veloso, B., Ribeiro, R.P., Gama, J. and Pereira, P.M. (2022) 'The MetroPT dataset for predictive maintenance', *Scientific Data*, 9, article 764.
- Williams, C.K.I. (2021) 'The effect of class imbalance on precision-recall curves', *Neural Computation*, 33(4), pp. 853-857.
- Zonta, T., da Costa, C.A., da Rosa Righi, R., de Lima, M.J., da Trindade, E.S. and Li, G.P. (2020) 'Predictive maintenance in the Industry 4.0: A systematic literature review', *Computers & Industrial Engineering*, 150, article 106889.